KRAQEL

KRAQEL's Recursive Authoring, Queries, Editions & Lore

Technisches Pflichtenheft · Revision 1 | Zielrelease 0.1.0 · Visual-Modul ab 0.2

Stand: 6. November 2026 | Status: Umsetzungsentwurf | Primärplattform: Windows

Ein quelloffenes, offlinefähiges Autorenstudio mit Manuskripteditor, Story-Bibel, Plotboard, Timeline, Git-Versionierung und austauschbarer KI-Anbindung. Neu in dieser Revision: eine projektweite Medienbibliothek und die Architektur für die spätere Generierung von Charakterbildern, Illustrationen, Karten und Symbolen.

1. Produktziele und Abgrenzung

• Schreiben zuerst: Romane, Drehbücher und Rollenspielmodule mit Projekten, Kapiteln und Szenen.

• Offline First: Schreiben, strukturieren, suchen und exportieren ohne Netz; lokale KI optional.

• Datensouveränität: Lokaler Projektordner, dokumentierte Datenformate, kein Accountzwang und keine Telemetrie ab Werk.

• Open Source: GPL-3.0-or-later als vorgesehene Lizenz, Abhängigkeiten vor Release prüfen.

• Visuelle Welt: Eigene und später KI-generierte Bilder werden referenzierbar, versionierbar und exportierbar.

Nicht in 0.1: Echtzeit-Kollaboration, Cloud-Synchronisation, mobile Clients, autonome Schreibagenten sowie fertige KI-Bildgenerierung und ein vollwertiger kartografischer Editor.

2. Technologie und Systemarchitektur

Schicht	Technik / Verantwortung
Desktop	Tauri 2 / Rust: Fenster, Dateisystem, IPC, Berechtigungsgrenzen
UI	React, TypeScript, Vite; Tailwind und zugängliche UI-Primitives
Editor	Tiptap / ProseMirror: Rich Text, Selektion, Undo/Redo
Daten	SQLite + FTS5: Transaktionen, Indizes, Volltextsuche
Visuell	React Flow / dnd-kit: Beziehungen, Karten des Plotboards; Kartenmodell später erweiterbar
Git	Rust-Service für deterministische Markdown-/JSON-Snapshots und Assets
KI	Getrennte Text- und Bildprovider, Capability-Registry, Opt-in für Cloud
Qualität	Vitest, Playwright, Rust-Tests; CI, Schema- und Migrationsprüfungen

Datenfluss: React nutzt typisierte Tauri-Commands; Rust führt Storage, Git, Dateisystem und externe Requests aus. SQLite ist die autoritative lokale Datenquelle. Projektdateien und Medien liegen im Projektordner; Git erfasst portable, deterministische Exporte und geeignete Assets, nicht die laufende SQLite-Datei.

3. Funktionale Anforderungen 0.1

ID	Modul	Nachweisbare Mindestfunktion
F01	Schreiben	Projekt-/Kapitel-/Szenenbaum, Rich Text, Autosave, Undo/Redo, Wortzählung, Fokusmodus
F02	Story-Bibel	Figuren, Orte, Fraktionen, Gegenstände, freie Eigenschaften, Beziehungen, Rückverweise
F03	Plotboard	Szenenkarten, Drag-and-drop, POV, Status, Handlungsstränge
F04	Timeline	Ereignisse, Zeitebenen, relative/absolute Ordnung, Szenenlinks
F05	Text-KI	OpenAI, Claude, Gemini, Mistral, Ollama; Modellwahl, Streaming, Kontextfreigabe
F06	Kontinuität	Regelbasierte Befunde und optionale KI-Hinweise mit Fundstellen
F07	Git	Init, Status, Diff, Commit und lesende Wiederherstellung aus Export-Snapshot
F08	Import/Export	Markdown, HTML, DOCX, PDF; definierte unterstützte Formatierungen
F09	Medien / Bildvorbereitung	Lokale Medienbibliothek, Entitätenzuordnung, visuelle Profile, Stilvorlagen, Prompt-Entwurf und Bild-Capabilities ohne aktiven Generator

Querschnittsregel: KI-Textvorschläge und Bildentwürfe werden nie ungefragt übernommen. Jede Änderung erfordert eine explizite Aktion.

4. Datenmodell (konzeptionell, SQLite)

Entität	Schlüsselattribute / Zweck
project / document	Projekt, Einstellungen, Schema-Version; Kapitel-/Szenenbaum, versioniertes Tiptap-JSON, Revision
entity / relation / link	Figuren, Orte, Fraktionen, Objekte, freie Felder; gerichtete Beziehungen und Rückverweise
storyline / timeline / event	Handlungsstränge, Zeitebenen, Ereignisse, Zeitpunkt / Sequenz, Verknüpfungen
ai_run / finding	Provider-/Laufstatus; Kontinuitätsbefunde mit Quellen und Schweregrad
media_asset (neu)	id, project_id, kind, relative_path, mime_type, checksum, width, height, provenance, created_at
asset_link (neu)	asset_id, target_type, target_id, role; n:m-Zuordnung zu Szenen, Figuren, Orten usw.
visual_profile (neu)	entity_id, traits_json, canonical_asset_id, exclusions_json; konsistente Merkmale
style_preset (neu)	project_id, name, visual_rules_json, applicable_kinds; projektweiter visueller Stil
generation_job (vorbereitet)	id, project_id, provider, model, prompt, settings_json, status, error, created_at; Laufzeit in 0.2

Technische Regeln: UUIDs, referenzielle Integrität, Fremdschlüssel, Migrationsversionen und relative Asset-Pfade. Dateinamen werden normalisiert; Lösch- und Umbenennungsoperationen müssen verlinkte Assets berücksichtigen. Roh-Prompts und Referenzbilder sind projektbezogene sensible Daten und werden nicht unbemerkt in Logs übertragen.

5. Text-KI, Bild-KI und Datenschutz

Textanbieter: OpenAI, Anthropic Claude, Google Gemini, Mistral und lokal Ollama. Späterer Bildbetrieb: passende Bild-APIs sowie lokale Workflows wie ComfyUI, sofern installiert. Modellfunktionen werden zur Laufzeit anhand einer Capability-Registry angezeigt, statt Fähigkeiten pauschal für alle Anbieter anzunehmen.

Capability	Bedeutung
text.generate / text.analyze	Formulieren, Überarbeiten, Kontinuitätsanalysen
structured.extract	Figuren, Orte, Ereignisse und visuelle Merkmale strukturiert extrahieren
image.generate	Neues Bild aus freigegebenem Kontext erzeugen (ab 0.2)
image.edit / reference	Vorhandenes Bild unter Erhalt von Referenzmerkmalen bearbeiten (ab 0.2, anbieterspezifisch)
image.map_render	Strukturierte Karte in grafische Ausgabe übertragen (nach funktionalem Kartenmodell)

Cloud-Zugriffe erfolgen nur nach expliziter Freigabe mit Vorschau von Texten, Medien und Metadaten. API-Schlüssel verbleiben im Betriebssystem-Schlüsselspeicher. Die Anwendung funktioniert ohne registrierte Provider; inaktive Fähigkeiten werden verständlich ausgewiesen. KI-generierte Ergebnisse erhalten eine Herkunftsangabe und können als Entwurf verworfen werden.

6. Modul V: Visuelle Assets und Generierung

V.1 Bestandteil der Version 0.1 (Muss)

• Lokale Medienbibliothek: Bilder importieren, anzeigen, verschlagworten, ersetzen und löschen; Miniaturansichten und Metadaten.

• Bilddateien mit Figuren, Orten, Szenen, Fraktionen, Objekten und Ereignissen verknüpfen; mehrere Varianten je Objekt.

• Visuelle Profile und projektweite Stilpresets, einschließlich expliziter Kernmerkmale und unerwünschter Abweichungen.

• Auswahl aus Story-Bibel und Szenentext zu einem editierbaren Bildprompt zusammenstellen; als Text speichern/exportieren.

• Provider-Registry um Bild-Capabilities und Generation-Job-Datenvertrag ergänzen, ohne Netzpflicht oder Bildgenerierung.

V.2 Erweiterung ab Version 0.2 (Soll)

• Generierung und Bearbeitung von Charakterporträts, Szenenillustrationen, Requisiten, Wappen und Symbolen mit auswählbaren Cloud- oder lokalen Bildmodellen.

• Referenzbilder, Stilvorgaben und konsistente Kernmerkmale pro Figur wiederverwenden; Bildvarianten nebeneinander vergleichen.

• Aufträge mit Status, Abbruch, Fehlerdetails und optionalen Seeds verwalten; technische Herkunfts- und Lizenzmetadaten hinterlegen.

• Manuelle Freigabe: Generierung ersetzt niemals automatisch ein kanonisches Referenzbild oder eine vorhandene Datei.

• Karten zweistufig erstellen: zuerst Orte/Kanten/Distanzen als nachvollziehbares logisches Modell, danach optional eine illustrative Darstellung. Keine ungeprüfte KI-Karte als geografisch korrekt ausgeben.

7. GitHub-Repository-Struktur

```
kraqel/
  .github/workflows/            CI, Releases, Sicherheitschecks
  apps/desktop/                 Tauri, Rust-Commands, React UI
  packages/domain/              Typen und Validierung
  packages/editor/              Tiptap-Erweiterungen
  packages/ui/                  UI-Komponenten
  packages/ai-contracts/        Text-/Bild-Capabilities, Provider-DTOs
  packages/visual-contracts/    Bildprompts, Stil- und Asset-Schemas (neu)
```

```
crates/storage/              SQLite, Migrationen, Medien-Repository
crates/git-service/          Git und Snapshots
crates/export/               Markdown, HTML, DOCX, PDF
crates/continuity/           Konsistenzregeln
crates/asset-service/        Import, Hash, Pfade, Thumbnails (neu)
docs/architecture/           ADRs, Threat Model
docs/specifications/         Pflichtenhefte, Abnahmekriterien
docs/data-format/            Offener Exportstandard
examples/                    Testprojekte und Beispielbilder
README.md  LICENSE  CONTRIBUTING.md  SECURITY.md
```

Git erfasst nur freigegebene, deterministische Snapshots einschließlich zulässiger Assets. Große oder lizenzrechtlich eingeschränkte Medien werden nach dokumentierter Regel behandelt. API-Schlüssel, private Manuskripte, temporäre KI-Dateien und die laufende SQLite-Datei sind ausgeschlossen.

8. Meilensteine

Phase	Liefergegenstand	Abnahmekriterium
M0	Monorepo, Tauri, Datenbank, CI, Berechtigungsmodell	Windows-Build läuft; Tests grün
M1	Editor, Szenenbaum, Autosave, Backup	Schreiben und Wiederherstellen fehlerfrei
M2	Story-Bibel, Plotboard, Timeline	Szenen/Entitäten/Zeiten verknüpfbar
M3	Git, Im-/Export, Medienbibliothek, visuelle Profile	Assets und Daten exportierbar, Backlink intakt
M4	Text-KI, Kontinuität, Prompt-Entwürfe, Bild-Contracts	Keine ungefragten Änderungen/Uploads
M5	Windows-Release 0.1, Tests, Dokumentation	Reproduzierbare Installation und Abnahme
M6 (0.2)	Bildgenerierung, Referenzkonsistenz, Bildvarianten, Kartenmodell	Manuelle Freigabe, nachvollziehbare Herkunft, Offline-Betrieb mit lokalem Provider

9. Erste Codex-Aufgaben / Issues

KQ-001 · Monorepo mit Tauri 2, React, TypeScript, Rust-Crates und Windows-CI anlegen.

KQ-002 · Domänenmodell, typisierte Commands und ADR zur SQLite-Autorität definieren.

KQ-003 · SQLite-Schema, Migrationen, FTS5 und Migrationstests implementieren.

KQ-004 · Lokalen Projektmanager inkl. Backup, Path-Safety und Recovery bauen.

KQ-005 · Tiptap-Editor mit Autosave, Undo/Redo, Szene-/Kapitelbaum integrieren.

KQ-006 · Story-Bibel mit Entitäten, Relationen, Rückverweisen und Suche bauen.

KQ-007 · Plotboard und parallele Timelines mit Zeitmodell umsetzen.

KQ-008 · Deterministische Projekt-Snapshots und Git-Status/Commit/Diff entwickeln.

KQ-009 · Markdown-/HTML-/DOCX-/PDF-Pipeline und Exporttests implementieren.

KQ-010 · Text-Provider-Interface und Ollama-Adapter mit Streaming bauen.

KQ-011 · OpenAI/Claude/Gemini/Mistral, Key-Schutz, Kontextvorschau anbinden.

KQ-012 · Kontinuitätsregeln, Quellenanker und KI-Diff-Review integrieren.

KQ-013 · Medienbibliothek implementieren: Asset-Pfade, Hash, Metadaten, Import, Vorschau, Links.

KQ-014 · Visual Profile und Style Preset samt CRUD und Migrationen umsetzen.

KQ-015 · Editierbaren Prompt-Builder aus Szenen- und Story-Bibel-Daten entwickeln.

KQ-016 · Capability-Registry für Bildprovider und Job-Vertragsmodelle definieren; Mock-Provider testen.

KQ-017 · 0.2: erste echte Bildpipeline, Variantenansicht, explizite Übernahme und Referenzprüfung.

KQ-018 · 0.2: strukturiertes Kartenmodell und optionalen illustrativen Renderer entwerfen.

10. Akzeptanztests und Qualitätsziele

• Offline: Alle 0.1-Funktionen einschließlich Medienverwaltung und Prompt-Entwurf ohne Internet nutzbar.

• Speicherintegrität: 100 Szenen und 50 verlinkte Bilder bleiben nach Neustart und Sicherung konsistent; fehlende Assets werden sichtbar gemeldet.

• Referenzen: Ein Charakterbild ist mehreren Szenen zuordenbar; Umbenennung löst keine gebrochenen Links aus.

• KI-Datenschutz: Bei Cloud-Opt-out verlassen weder Manuskripttexte noch Bilder oder Metadaten den Rechner.

• Timeline: Mindestens zwei Zeitebenen mit getrennter Chronologie und Erzählreihenfolge möglich.

• Git: Export-Snapshot ist bei unverändertem Projekt byte-stabil, Assets erhalten Hash-Referenzen.

• Export: Definierte Textformatierungen und Mediendateien sind im dokumentierten Austauschformat nachvollziehbar.

• 0.2-Bildtest: Generierung über einen lokalen oder bewusst freigegebenen Cloudprovider; bestehende Assets werden ohne explizite Übernahme nicht überschrieben.

11. Offene Entscheidungen und rechtlicher Hinweis

• Projektpaket: Verzeichnis (SQLite + assets + manifest) oder zusätzlich gepacktes Austauscharchiv.

• Umfang von DOCX-Roundtrip, Drehbuchformatierung und Medien-Einbettung je Exportformat festlegen.

• Für 0.2 Anbieterbedingungen, Nutzungsrechte, Modelllizenzen und Kennzeichnung erzeugter Medien prüfen.

• Projektname KRAQEL laut Projektinitiator*in beim DPMA geprüft; EU-Rechte und ähnliche ältere Kennzeichen bleiben gesondert abzuklären.

Versionshinweis: Revision 1 erweitert den ursprünglichen 0.1-Entwurf um F09, das visuelle Datenmodell und die Bild-KI-Roadmap. Die eigentliche Bildgenerierung wird bewusst erst für 0.2 als Umsetzungsziel geführt.